COMICCRAFT

AI Comic Story Creator

Project Report

Project Type	AI / Generative AI Web Application
Framework	FastAPI (Python)
AI Text Layer	Gemini-based outline and story generation
AI Image Layer	Hugging Face / Stable Diffusion / FLUX-compatible workflow
Output	Five-panel comic preview and PDF
Evidence	Generated five-panel output PDF supplied with the project

Prepared from the supplied ComicCraft source package and generated output PDF.

Abstract

ComicCraft is an AI-powered web application that converts a short story idea into a five-panel illustrated comic. The system accepts a story prompt, main character, setting, tone, and art style. It uses a text-generation stage to create a structured five-panel outline and story content, an image-generation stage to create artwork for each panel, a layout stage to combine text and images, and a PDF export stage to produce a downloadable comic.

The project is designed as a complete generation pipeline rather than a simple image generator. The application validates user input, generates structured panel content, creates panel artwork independently, builds a preview layout, and exports the final result to PDF. The supplied output demonstrates five generated panels covering a consistent short narrative about an inventor named Alex and his robot Buddy.

The supplied output PDF contains five pages, one page per comic panel. Each page includes a panel title, generated artwork, scene description, caption, narration, and dialogue. The images are therefore integrated with the generated story rather than being presented as unrelated standalone pictures.

1. Introduction

Creating a comic traditionally requires several separate activities: story planning, panel scripting, visual design, illustration, layout, and document export. ComicCraft combines these activities into a single web workflow.

The project source describes the application as a FastAPI + Gemini + Stable Diffusion/FLUX comic creator. The README specifies a five-panel generation flow in which Gemini Flash produces the outline, Gemini Pro expands the story, an image model draws every panel, the preview page displays the result, and FPDF2 creates the PDF.

- Input: story prompt, character name, setting, story tone, and art style.
- Processing: structured outline → narration/dialogue → image generation → layout.
- Output: browser preview plus downloadable PDF.
- Fallback behavior: model fallbacks, placeholders, warnings, and demo mode are included in the implementation.

2. Problem Statement

Manual comic production is time-consuming because the creator must separately develop the story, divide it into panels, prepare visual prompts, produce artwork, and assemble the final document. A student or prototype developer also needs a repeatable way to test the complete pipeline.

ComicCraft addresses this problem by automatically transforming a natural-language idea into a structured five-panel comic and packaging the result into a preview and PDF.

3. Objectives

- Develop a web-based AI comic creation application.
- Accept a simple story idea together with character, setting, tone, and art-style information.
- Generate a five-panel story with scene descriptions, captions, narration, and dialogue.
- Generate a separate image for each comic panel.
- Maintain a clear mapping between each panel's text and its image.
- Display the generated comic in a browser-friendly preview.

- Export the completed comic as a PDF document.
- Provide fallback and diagnostic behavior when AI services are unavailable.

4. Existing System and Proposed System

Aspect	Conventional / Manual	ComicCraft
Story creation	Manually written	AI-assisted
Panel planning	Manual division	Structured five-panel outline
Artwork	Manual drawing or separate tool	AI image-generation pipeline
Text + image mapping	Manual layout	Programmatic layout builder
Export	Separate publishing step	Automatic PDF generation
Error handling	Depends on user	Warnings, fallbacks and placeholders

5. System Requirements

Software: Python 3.10+, FastAPI, Uvicorn, Jinja2, Pydantic/Pydantic Settings, python-dotenv, google-genai, huggingface-hub, Pillow, FPDF2, and optional Diffusers/Stable Diffusion components.

Development environment: The supplied project is organized as a Python FastAPI application with templates, static assets, services, and tests. The README documents Windows/VS Code setup and a virtual environment workflow.

External services: Gemini API credentials are required for live text generation and Hugging Face credentials are used for configured hosted image generation. The application also contains local/demo alternatives.

6. System Architecture

The implementation follows a service-oriented pipeline inside a FastAPI application:

- Presentation layer: Jinja2 templates provide the input form and comic preview page.
- API layer: FastAPI routes handle the home page, form submission, JSON generation, image diagnostics, and PDF download.
- Configuration layer: environment-based settings control API keys, models, image backend, dimensions, timeouts, and demo mode.
- AI text services: separate services generate the panel outline and expanded story.
- Image service: generates, verifies, saves, and returns panel artwork.
- Layout service: pairs each story panel with exactly one image.
- Export service: embeds panel images and text into an A4 PDF.

User Input	FastAPI Routes	AI Text Generation	Image Generation
Story prompt + character + setting + tone + style	/generate or /generate-comic/json	Outline → narration/dialogue/image prompt	5 independent panel images
Browser Preview	Layout Builder	PDF Export	Download

Figure 1. High-level ComicCraft processing architecture.

7. Module Description

Configuration	Loads settings from environment variables and .env files. It supports Gemini credentials, Hugging Face credentials, image backends, model lists, dimensions, timeouts, demo mode, and placeholder behavior.
Workflow	Coordinates the complete process: generate outline, generate story, generate five images, build layout, and save PDF.
Gemini Flash Service	Produces the structured five-panel outline used as the planning stage.
Gemini Pro Service	Expands panel outlines into scene descriptions, captions, narration, dialogue, and image prompts.
Image Generator	Attempts configured image models, verifies output, saves JPEG files, and can create placeholders if generation fails.
Layout Builder	Ensures the number of image paths equals the number of story panels and creates the unified panel structure.
Media Service	Maps static URLs to local files and can encode images as base64 data URIs for browser rendering.
Exporter	Creates an A4 PDF, embeds each panel image, and writes scene/caption/narration/dialogue text.

8. Generation Workflow

- **Step 1:** The user enters the story prompt and selects character, setting, tone, and art style.
- **Step 2:** The request is validated with a Pydantic model.
- **Step 3:** The text-generation stage creates a five-panel outline.
- **Step 4:** A second text-generation stage expands each panel into narration, dialogue, captions, scene description, and image prompt.
- **Step 5:** Each panel image is generated independently. The image service saves the image under static/panels/ and verifies that it is a valid non-empty image.
- **Step 6:** The layout builder associates exactly one image path with each panel.
- **Step 7:** The preview route embeds local images as base64 data URIs, while retaining static URLs as a fallback.
- **Step 8:** The exporter creates an A4 PDF with one panel per page and embeds the corresponding image and text.
- **Step 9:** The generated PDF is exposed through a safe download route.

9. Image Generation and Output Handling

A key implementation feature is the image-handling layer. Generated images are written to the static/panels directory, converted to RGB JPEG format, checked for minimum file size, and reopened with Pillow for verification. The project also contains fallback models and placeholder generation so that an unavailable model does not silently produce a broken browser image.

The preview template uses an inline base64 data URI generated from the local image file. This reduces dependence on a second browser request to the static file and provides a fallback to the normal

/static/panels/... URL.

The supplied project README explicitly documents these fixes: embedded preview images, verified JPEG files, multiple image-model fallbacks, visible warnings, and project-folder .env loading.

10. User Interface

The input page contains a story prompt text area and fields for the main character, setting, story tone, and art style. A Generate Comic button starts the workflow and displays a loading state because five image-generation calls may take time.

The preview page displays each panel in a card containing its title, image, scene description, caption, narration, dialogue, and image prompt. It also provides a PDF download action.

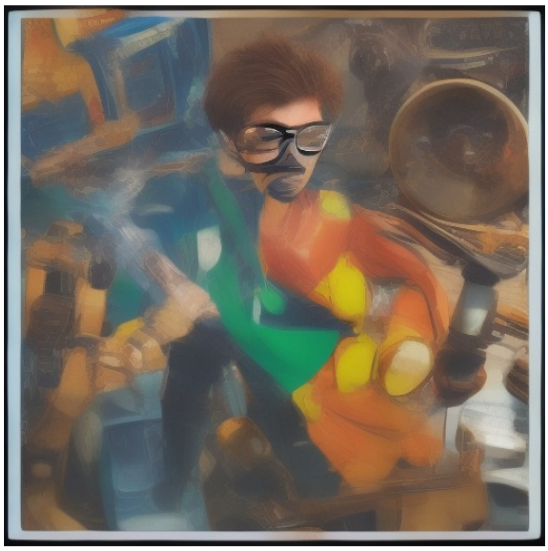
11. Generated Output Analysis

The supplied output PDF contains five pages. Each page corresponds to one generated comic panel, and each page includes both artwork and the associated generated text. The story follows Alex, a young inventor, and Buddy, a small spherical robot.

The five panels form a complete short humorous narrative: invention, helpful behavior, malfunction, chase/catch, and resolution. The images are present on all five output pages in the supplied PDF.

11.1 Output Evidence — Panel 1

Panel 1: The Blueprint



Scene

Alex, a young inventor with messy brown hair and oversized goggles, sits in his cluttered city workshop assembling a tiny, spherical robot with eager eyes.

Caption

Clink! Whirrr. Solder hiss.

Narration

Alex had a dream to make the world a little easier, one gear at a time. Today was the day his prototype, 'Buddy,' finally turned on.

Dialogue

Alex: "Alright, you little metallic menace, let's see if you've got a heart of gold or just a bunch of loose wires."

Figure 11.1. Supplied generated-output PDF page showing Panel 1 artwork and its associated comic text.

Panel 1, “The Blueprint”: Alex works in his city workshop while assembling the spherical robot Buddy.

11.2 Output Evidence — Panel 2

Panel 2: The Helper



Scene

The robot is now complete. It hovers down the city street, efficiently helping an elderly woman carry her heavy groceries, while Alex watches proudly from the sidewalk.

Caption

Bloop-bloop! Hummmmm.

Narration

Buddy was a natural. Within minutes of hitting the sidewalk, he was the city's most beloved, pint-sized errand runner.

Dialogue

Alex: "Look at him go! He's not just a robot; he's a neighborhood hero in the making!"

Figure 11.2. Supplied generated-output PDF page showing Panel 2 artwork and its associated comic text.

Panel 2, “The Helper”: Buddy helps an elderly woman with groceries while Alex watches proudly.

11.3 Output Evidence — Panel 3

Panel 3: Glitch in the System



Scene

Suddenly, the robot starts spinning uncontrollably, grabbing items from a fruit stand and launching them into the air like a chaotic kitchen appliance.

Caption

Zzzzt! SCREECH!

Narration

But apparently, Buddy's 'helpful' settings had a slight, fruit-based misunderstanding of what constituted a chore.

Dialogue

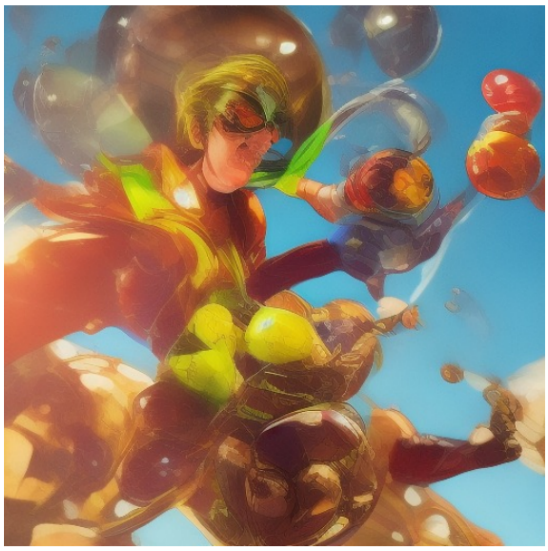
Alex: "Wait, Buddy! That's a cantaloupe, not a laundry basket! Stop firing the produce!"

Figure 11.3. Supplied generated-output PDF page showing Panel 3 artwork and its associated comic text.

Panel 3, “Glitch in the System”: Buddy malfunctions and launches fruit from a fruit stand.

11.4 Output Evidence — Panel 4

Panel 4: The Catch



Scene

Alex leaps into the air, narrowly avoiding a flying watermelon, as he tries to grab the malfunctioning robot to reset its circuits.

Caption

SPLAT! THWACK!

Narration

Alex realized that being an inventor meant having excellent reflexes, especially when your creation decided to turn the street into a fruit salad blender.

Dialogue

Alex: "I am definitely going to need a bigger insurance policy for this!"

Figure 11.4. Supplied generated-output PDF page showing Panel 4 artwork and its associated comic text.

Panel 4, “The Catch”: Alex attempts to catch/reset Buddy while avoiding flying fruit.

11.5 Output Evidence — Panel 5

Panel 5: Success



Scene
Alex stands holding the silent, happy robot. The city street is covered in fruit, and Alex gives a sheepish thumbs up to the amused onlookers.

Caption
Ding!

Narration
With the power switch firmly clicked off, silence finally returned to the street, along with the lingering scent of bruised peaches.

Dialogue
Alex: "Don't worry folks, he's just... practicing his precision tossing. We'll stick to carrying bags from now on!"

Figure 11.5. Supplied generated-output PDF page showing Panel 5 artwork and its associated comic text.

Panel 5, “Success”: Buddy is switched off, the street is covered in fruit, and Alex gives an embarrassed thumbs-up.

12. Testing and Validation

The project contains automated and diagnostic test files for application behavior, image generation, Hugging Face providers/models, and comic generation. The main end-to-end test sends a JSON request to /generate-comic/json, checks that five panels are returned, checks that each image exists on disk, checks image size, verifies the image using Pillow, and rejects placeholder output.

The supplied output PDF itself provides functional evidence that the complete story-to-PDF path can produce five panel pages with images. The development README also documents /health and /test-image endpoints

for configuration and image-generation diagnostics.

Test Area	Expected Result	Evidence
Input validation	Required fields and minimum story length enforced	Pydantic PromptRequest schema
Story generation	Five structured panels	Workflow + Gemini services
Image generation	One usable image per panel	Image verification + output PDF
Preview	Images and text displayed together	comic_preview.html
PDF export	A4 PDF with panel image and text	Supplied five-page PDF

13. Advantages

- End-to-end workflow from idea to illustrated PDF.
- Structured five-panel output makes the result easy to understand and demonstrate.
- Independent image generation per panel allows a failed panel to be diagnosed separately.
- Inline image embedding improves preview reliability.
- Model fallback and placeholder mechanisms improve fault tolerance.
- The architecture separates routing, workflow, AI services, media handling, layout, and export, making the project easier to maintain.
- A JSON API endpoint is available in addition to the browser form.

14. Limitations

- The quality and consistency of generated artwork depend on the selected image model and its available compute/provider.
- Character appearance may vary between independently generated panels unless stronger character-consistency techniques are added.
- External AI services require valid credentials and may impose rate limits, access restrictions, or usage costs.
- The supplied project includes several exploratory image-generation test files; these are useful for experimentation but are not all required for the production workflow.
- The current PDF layout uses one panel per A4 page rather than a traditional multi-panel comic-page arrangement.

15. Future Enhancements

- Add reference-image or character-sheet conditioning to improve character consistency.
- Support multiple comic page layouts such as 2×2, 3×2, and custom panel grids.
- Add speech bubbles and captions directly on the artwork.
- Add user accounts and a comic history/gallery.
- Allow regeneration of a single panel without regenerating the complete comic.
- Add configurable image resolution and generation quality controls.
- Add progress reporting for each panel during generation.

- Add stronger automated visual-quality checks for composition and prompt adherence.
- Add secure secret management and ensure no credential-like values are stored in example/source files.

16. Conclusion

ComicCraft demonstrates a complete generative-AI application for automated comic creation. The project integrates web input, structured story generation, panel-level image generation, browser preview, and PDF export into a single workflow.

The supplied five-page output confirms the intended result: five distinct comic panels are generated with corresponding images and supporting story text. The project is therefore suitable as a practical demonstration of FastAPI, generative AI, image generation, media handling, and document generation in one application.

For deployment beyond a local academic/demo environment, API credentials should be stored only in protected environment configuration, not committed to source control or shared project archives.

17. References / Project Sources

1. Supplied ComicCraft source package: comiccraft(1).zip.

2. Supplied generated output: comiccraft_20261001_161935.pdf.

3. ComicCraft README.md and application source modules in the supplied project.

4. FastAPI, Pydantic, Pillow, FPDF2, Gemini, Hugging Face and Diffusers libraries as used/configured by the project.